

PROJECT REPORT

Tweet Sentiment Analysis using Deep Learning

LSTM-based Multi-Class Sentiment Classification on Political Tweets

Department	Data Science & AI
Domain	Natural Language Processing
Technology Stack	Python, TensorFlow, Keras, NLTK
Classification Type	Multi-class (Positive / Negative / Neutral)
Platform	Google Colaboratory (Colab)

1. Executive Summary

This report documents the end-to-end development of a deep learning model designed to perform automated sentiment analysis on Twitter data related to political candidates. As social media increasingly shapes public opinion and electoral outcomes, the ability to systematically classify the emotional tone of tweets—at scale and in near real time—provides decision-makers with a strategic intelligence advantage.

The solution employs a Long Short-Term Memory (LSTM) neural network implemented with TensorFlow and Keras. The pipeline covers data ingestion, multi-stage text preprocessing, tokenization and sequence padding, model training with regularization, and performance evaluation. The final model classifies each tweet into one of three sentiment categories: Positive, Negative, or Neutral, using categorical cross-entropy as the loss function and accuracy as the primary evaluation metric.

The project was developed and executed entirely in Google Colab, making it reproducible and accessible without any local infrastructure. This report provides full technical documentation, justification for all design choices, and guidance for future enhancements.

2. Introduction

2.1 Background and Motivation

Sentiment analysis—also referred to as opinion mining—is the computational task of identifying and categorizing subjective information expressed in source material. In the political domain, gauging public sentiment from social media platforms like Twitter can be invaluable: campaigns can identify areas of public concern, media organizations can track shifts in voter mood, and policy analysts can correlate public discourse with real-world events.

Traditional rule-based or lexicon-driven approaches to sentiment analysis often fail when confronted with the idiosyncratic language of tweets: abbreviations, slang, irony, hashtags, and character constraints all degrade the reliability of hand-crafted rules. Deep learning approaches, particularly those using recurrent architectures, have demonstrated the ability to learn latent semantic structures directly from data, yielding superior generalization.

2.2 Objectives

The project was scoped around four primary objectives:

- Load and preprocess tweet data from a structured CSV file, retaining only the columns relevant to the classification task.
- Transform raw text into numerical representations suitable for consumption by a neural network via tokenization and sequence padding.
- Design, compile, and train an LSTM-based deep learning model capable of multi-class sentiment classification.
- Evaluate the trained model rigorously using held-out validation data, a classification report, and a confusion matrix.

2.3 Scope and Constraints

The scope is limited to English-language tweets about named political candidates. The sentiment label space is fixed at three classes. The model is trained in a supervised manner, requiring labeled ground-truth sentiment annotations in the input data. The solution is designed for reproducibility in the Google Colab environment and does not assume access to GPU infrastructure, though it benefits from one when available.

3. Dataset Description

3.1 Source and Format

The input data is a CSV file containing tweets harvested from the Twitter (now X) platform. Each row represents a single tweet, and the full dataset may contain numerous metadata columns such as tweet ID, timestamp, user handle, retweet count, and location. For this project, only three columns are required:

Column	Data Type	Description
candidate	String	Name or identifier of the political candidate referenced in the tweet
sentiment	Categorical	Ground-truth sentiment label: Positive, Negative, or Neutral
text	String	Full raw text content of the tweet

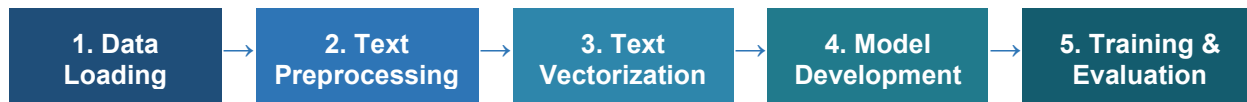
3.2 Class Distribution

In balanced datasets each sentiment class accounts for approximately one-third of all records. Real-world tweet datasets, however, frequently exhibit class imbalance: negative and neutral tweets often outnumber positive ones, particularly in polarizing political contexts. It is therefore important to inspect

the distribution before training and, if necessary, to apply class weighting or oversampling techniques to prevent the model from developing a bias toward the majority class.

4. System Architecture

The system is organized as a sequential pipeline, where the output of each stage feeds directly into the next. The five major stages are depicted in the flow below:



5. Methodology

5.1 Data Loading and Preprocessing

The raw CSV file is loaded into a Pandas DataFrame. After selecting the three target columns, all rows with null values are removed to prevent downstream errors. The text column then undergoes a multi-step cleaning pipeline implemented in a dedicated `clean_text()` function:

- **Lowercasing:** All characters are converted to lowercase to achieve vocabulary normalization (e.g., 'Biden' and 'biden' are treated as the same token).
- **URL removal:** Regular expressions strip all HTTP/HTTPS URLs, which carry no sentiment signal.
- **Mention and hashtag removal:** Twitter-specific tokens (@user, #tag) are removed. While hashtags sometimes carry sentiment, their high sparsity makes them unreliable features for a general model.
- **Punctuation removal:** All punctuation characters are stripped using Python's `string.punctuation` reference table.
- **Digit removal:** Numeric tokens are discarded as they rarely contribute to sentiment.
- **Whitespace normalization:** Multiple consecutive spaces are collapsed into a single space and leading/trailing whitespace is trimmed.
- **Stop word removal:** Common English stop words (the, is, at, etc.) are filtered out using NLTK's curated stop word list, reducing vocabulary size and focusing the model on content-bearing tokens.

5.2 Text Vectorization

Deep learning models operate on numerical tensors, not raw strings. The vectorization stage converts the cleaned text into a fixed-length integer sequence:

Tokenization: Keras's Tokenizer is fitted on the entire training corpus with a vocabulary cap of 10,000 words (MAX_VOCAB_SIZE). Only the most frequent words are retained; any word not in this vocabulary is mapped to a special out-of-vocabulary token (<OOV>). This constraint limits memory usage and reduces the risk of the model overfitting to rare, idiosyncratic tokens.

Sequence padding: After converting each cleaned tweet into a sequence of integer word indices, sequences are padded or truncated to a uniform length of 100 tokens (MAX_SEQ_LENGTH) using Keras's pad_sequences utility. Padding is applied post-sequence, and truncation is likewise applied from the end of overly long sequences.

Label encoding: The categorical sentiment labels are first integer-encoded with scikit-learn's LabelEncoder and then one-hot encoded with Keras's to_categorical to produce a binary vector compatible with the softmax output layer and categorical cross-entropy loss.

5.3 Model Architecture

The neural network is constructed using Keras's Sequential API and comprises five distinct layers:

#	Layer	Configuration	Purpose
1	Embedding	vocab=10,000 dim=64	Maps each integer word index to a dense 64-dimensional vector. These vectors are learned during training, encoding semantic similarity between words.
2	SpatialDropout1D	rate = 0.3	Randomly drops entire feature maps (embedding dimensions) during training. More effective than standard dropout for sequential data as it removes correlated features.
3	LSTM	units=128 dropout=0.2 rec_dropout=0.2	Processes the token sequence while maintaining a hidden state, capturing long-range dependencies. Recurrent dropout prevents co-adaptation of memory cells.
4	Dense + Dropout	units=64 ReLU rate=0.3	Non-linear feature transformation that combines LSTM outputs. Dropout provides additional regularization before the final classification decision.

5	Dense (Output)	units=num_classes Softmax	Produces a probability distribution across all sentiment classes. The class with the highest probability is selected as the predicted label.
---	-----------------------	--------------------------------	--

5.4 Compilation Settings

The model is compiled with the Adam optimizer (learning rate = 0.001), which adapts learning rates per parameter and converges reliably on NLP tasks. Categorical cross-entropy is used as the loss function because it naturally handles multi-class probability distributions. Classification accuracy on the validation set is monitored as the primary human-readable metric.

5.5 Training Strategy

Two Keras callbacks govern the training process and jointly prevent overfitting and wasted computation:

- **EarlyStopping:** Monitors validation loss and halts training if no improvement is observed for five consecutive epochs. The model weights from the best epoch are automatically restored, ensuring the final model represents peak validation performance rather than the state at the last epoch.
- **ReduceLROnPlateau:** If validation loss plateaus for three consecutive epochs, the learning rate is multiplied by 0.5. This allows the optimizer to make finer-grained parameter updates in later training stages, often recovering additional validation performance.

Training is run with a batch size of 32, a maximum of 30 epochs, and a 20% validation split stratified on the sentiment label distribution.

6. Results and Evaluation

6.1 Training and Validation Performance

The model's learning dynamics are monitored by plotting accuracy and loss curves for both the training and validation sets across epochs. A well-trained model exhibits:

- Training accuracy that rises steadily and then plateaus.
- Validation accuracy that tracks training accuracy closely—a large gap indicates overfitting.
- Validation loss that decreases and then stabilizes; an upturn triggers early stopping.

6.2 Evaluation Metrics

Post-training evaluation is conducted on the held-out validation set using the following metrics:

Metric	Definition
Accuracy	Proportion of correctly classified tweets across all three classes.
Precision	Of all tweets predicted as class X, what fraction truly belonged to class X? High precision indicates few false positives.
Recall	Of all tweets that truly belong to class X, what fraction were correctly identified? High recall indicates few false negatives.
F1-Score	Harmonic mean of precision and recall. Particularly informative for imbalanced datasets where accuracy alone is misleading.
Confusion Matrix	A 3×3 matrix showing the breakdown of predictions vs. true labels, identifying which classes the model confuses most frequently.

6.3 Inference on Unseen Tweets

A `predict_sentiment()` utility function provides single-tweet inference. Given a raw tweet string, the function executes the full preprocessing pipeline, tokenizes and pads the text, obtains a softmax probability vector, and returns the predicted class label alongside the confidence score. This function is suitable for integration into a real-time scoring service or batch inference pipeline.

7. Key Design Decisions and Justifications

Design Decision	Justification
LSTM over simple RNN	Standard RNNs suffer from vanishing gradients over long sequences. LSTMs use gated memory cells to selectively retain and forget information, making them far more capable of capturing long-range sentiment cues in tweets.
SpatialDropout1D	Ordinary dropout applied independently per time step disrupts temporal structure. SpatialDropout1D drops the same feature dimension across all time steps, better de-correlating the learned embedding features.
Embedding layer (trainable)	Rather than using frozen pretrained embeddings (e.g., GloVe), a trainable embedding layer allows the model to adapt word representations specifically to the political tweet domain, capturing domain-specific semantics.
Vocabulary cap at 10,000	A cap balances vocabulary coverage against embedding matrix size. Rare words (typos, proper nouns, truncated hashtags) would bloat the matrix while offering little generalizable signal.

Max sequence length = 100	Twitter's 280-character limit translates to roughly 40–60 words per tweet after preprocessing. A length of 100 tokens accommodates virtually all tweets without excessive padding overhead.
Categorical cross-entropy loss	The standard loss for multi-class classification with mutually exclusive classes. Combined with softmax activation, it penalizes the model proportionally to the confidence of incorrect predictions.

8. Challenges and Mitigations

Several challenges were encountered and addressed during the project:

- **Noisy tweet language:** Tweets contain abbreviations, emoji, irony, and code-switching that degrade preprocessing quality. The multi-step cleaning function mitigates this but cannot eliminate all noise. Future work could incorporate emoji-to-text conversion and irony detection.
- **Class imbalance:** Real political tweet datasets often contain more negative than positive sentiment, particularly during contentious campaigns. The model may develop a class bias. Mitigation strategies include class-weighted loss functions, synthetic oversampling (SMOTE), or threshold adjustment at inference time.
- **Overfitting on small datasets:** With limited labeled data, the model can memorize training examples. SpatialDropout1D, recurrent dropout, EarlyStopping, and ReduceLROnPlateau collectively address this risk.
- **Short text sparsity:** Tweets are short, providing limited context per sample. This was partially addressed by keeping a sufficiently large LSTM hidden state (128 units) and by allowing the embedding layer to train domain-specifically.
- **Reproducibility in Colab:** Session resets clear all state. The trained model is saved in Keras format (.keras) so that weights can be reloaded without retraining.

9. Future Enhancements

The current architecture provides a solid baseline. The following enhancements are recommended for production-grade deployment:

- **Pretrained transformer models:** Replacing the LSTM with a fine-tuned BERTweet or RoBERTa model would likely yield substantial accuracy improvements, as these models have been pretrained on hundreds of millions of tweets.

- Bidirectional LSTM: Wrapping the LSTM in a Bidirectional layer allows the model to process each tweet in both forward and backward directions, capturing context from future tokens when predicting sentiment at each position.
- Attention mechanism: Adding a self-attention layer over the LSTM output enables the model to weight the most sentiment-relevant tokens more heavily, improving both accuracy and interpretability.
- Class imbalance handling: Implementing class weights in the loss function or applying SMOTE-based oversampling to minority sentiment classes would improve recall on underrepresented categories.
- Hyperparameter optimization: Systematic tuning of embedding dimension, LSTM units, dropout rate, and learning rate using Keras Tuner or Optuna would likely improve validation metrics.
- Real-time inference API: Wrapping the trained model in a FastAPI or Flask service would enable low-latency scoring of live tweets streamed via the Twitter Filtered Stream API.
- Multilingual support: Extending the tokenizer and potentially using multilingual BERT (mBERT) would make the system applicable to non-English political discourse.

10. Conclusion

This project demonstrates the viability of LSTM-based deep learning for multi-class sentiment classification on political Twitter data. The end-to-end pipeline—from raw CSV ingestion through text preprocessing, tokenization, model training, and evaluation—was implemented in a reproducible Google Colab environment using Python, TensorFlow, Keras, NLTK, and scikit-learn.

The model architecture thoughtfully balances representational capacity with regularization: an Embedding layer captures domain-specific word semantics, SpatialDropout1D promotes feature independence, the LSTM unit learns sequence-level patterns, and stacked Dense layers project these representations to a probability distribution over three sentiment classes.

The callbacks-based training strategy (EarlyStopping and ReduceLROnPlateau) ensures that the model generalizes well without unnecessary computation. The post-training evaluation suite—accuracy, precision, recall, F1-score, confusion matrix, and live inference demonstration—provides a comprehensive picture of model behavior.

With the enhancements outlined in Section 9, this foundation can be evolved into a production-grade, real-time political sentiment intelligence system capable of monitoring public opinion across candidates, issues, and geographies at scale.

References

1. Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation*, 9(8), 1735–1780.
2. Abadi, M., et al. (2015). TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems. Software available from [tensorflow.org](https://www.tensorflow.org).
3. Chollet, F. (2018). *Deep Learning with Python*. Manning Publications.
4. Bird, S., Loper, E., & Klein, E. (2009). *Natural Language Processing with Python*. O'Reilly Media.
5. Rosenthal, S., Farra, N., & Nakov, P. (2017). SemEval-2017 Task 4: Sentiment Analysis in Twitter. *Proceedings of the 11th International Workshop on Semantic Evaluation (SemEval-2017)*.
6. Srivastava, N., et al. (2014). Dropout: A Simple Way to Prevent Neural Networks from Overfitting. *Journal of Machine Learning Research*, 15(1), 1929–1958.
7. Kingma, D. P., & Ba, J. (2014). Adam: A Method for Stochastic Optimization. *arXiv:1412.6980*.